# Agentic AI for Accounts Payable

Invoice three-way match with a team of cooperating agents

---

**Authors:** Swastik Gavhane, Saafalya Sengupta

**PRN:** 24070126224, 24070126158

**Institution:** Symbiosis Institute of Technology

**Domain:** Banking & Finance, Business Operations

**Source code:** https://github.com/Saafalya/invoice_agents

## 1. Abstract

Before paying a vendor, finance teams must confirm that the invoice agrees with the purchase order (what was ordered) and the goods receipt (what arrived). Done by hand this is slow and repetitive, and every miss becomes an overpayment, a duplicate payment, or a payment to a fraudster. This project builds a team of seven cooperating AI agents that check each invoice against its purchase order and goods receipt, look for fraud signals, and attach a written reason to every decision. Risky invoices pause for a human reviewer, and a long-term vendor memory learns from every decision. Invoices can be loaded as a batch or uploaded as PDFs. An optional local language model (Ollama) makes the output easier to act on: it explains each fraud score in plain words, gives the Critic a second opinion on every extraction, and writes a short case summary for each escalated invoice. The model never makes the decision, and any figure it invents is rejected before a reviewer sees it.

The system was validated on 24 synthetic invoices containing 8 seeded errors, 3 of them fraud. It caught all 8, identified every issue type correctly, wrongly flagged none of the 16 clean invoices, and flagged INR 0.99 lakh at risk, which equals the seeded total. A larger run over 37,200 generated invoices decided 99.98% correctly. These results come from rule-based matching on known data and should be read as proof that the pipeline works, not as a measure of real-world accuracy.

## 2. Problem Statement

Three-way match requires agreement between three documents:

| Document | What it records | Question it answers |
|---|---|---|
| Purchase order (PO) | What we agreed to buy | What did we order? |
| Goods receipt (GRN) | What actually arrived | What did we receive? |
| Invoice | What the vendor bills | What are we being asked to pay? |

All three must agree before payment is released. Errors that commonly slip through include:

- **Price above the PO:** the invoice charges a price above the agreed PO price;
- **Billed more than received:** the vendor bills for more units than the goods receipt shows;
- **Duplicate payment:** the same invoice is submitted and paid more than once;
- **Fraud:** a fake vendor copies a real PO, or a real vendor asks to be paid into a new bank account.

# 3. From Manual Checks to Agents

| Today: manual review | With agents |
|---|---|
| Staff compare three documents by hand | Planner orders the batch by urgency |
| Slow and repetitive for every invoice | Agents extract, validate and match |
| Errors often found after payment | Problems caught before payment |
| No consistent written reason | Every decision comes with a reason |

Humans are not removed from the process: invoices the agents escalate stop and wait in a review queue, where a person approves, holds or rejects them.

# 4. System Design

## 4.1 The seven agents

Each agent has a single responsibility.

| Agent | Responsibility |
|---|---|
| Planner | Sequences the batch by due date and amount and flags urgent invoices. |
| Extraction | Turns invoice text into a typed schema using an LLM or a parser; retries if the result is rejected. |
| Critic | Rejects extractions that fail arithmetic checks or contain values not found in the document, asks the local model for a second opinion, and confirms that every variance is explained. |
| Matching | Compares invoice, PO and goods receipt on price, quantity, tax, vendor and duplicates. |
| Fraud | Flags lookalike vendor names, bank-account changes, amount spikes and round totals, and gives each invoice a 0-100 risk score with reasons. The local model turns the score into a plain-language note. |
| Decision | Applies business rules to approve, hold or escalate, writes the reason, and prepares a case summary for every escalated invoice. |
| Human Review | Pauses escalated invoices for a person (LangGraph interrupt), resumes with their answer, and writes the outcome to vendor memory. |

### 4.2 Agent hand-off and retry loop

The agents run in this order: Planner, Extraction, Critic (validate), Matching (three-way match), Fraud, Critic (cross-check), Decision, Human Review. The Critic therefore appears twice in the workflow.

The retry loop is a key agentic behaviour. If the Critic rejects an extraction, the Extraction Agent retries with a more robust parser. If it still fails, the invoice is escalated. The Critic also runs a grounding check: the total, invoice number, vendor, PO number and bank account must all appear in the document, so neither the parser nor a language model can invent a value. When Ollama is switched on, the model gives a second opinion on the extraction. Its doubt triggers one more retry with the parser, but on its own it never escalates an invoice.

### 4.3 Decision outcomes

| Outcome | When it applies |
|---|---|
| Approved | No issues. Safe to pay. |
| Hold | Low-value mismatch. The vendor is asked to correct it. |
| Escalated | Fraud signal (lookalike vendor, bank change, risk score 40+), duplicate, unknown vendor, or INR 10,000 or more at risk. The invoice pauses for a human. |
| Rejected | Set only by a human reviewer, for an escalated invoice that must not be paid. |

### 4.4 Architecture

The system is organised in layers:

- **User interface:** a ledger-style app with a live agent trace, a seven-step pipeline strip, a review queue of case cards, PDF upload, a results dashboard and a vendor ledger (Streamlit).
- **Orchestration:** a state graph with shared state, a conditional retry edge, an interrupt with a checkpointer for human review, and a plain-Python fallback (LangGraph).
- **Agents:** Planner, Extraction, Critic, Matching, Fraud, Decision and Human Review.
- **Tools and data:** PO database, GRN database, vendor master, duplicate registry, vendor memory and business rules.
- **LLM:** an Ollama local model, or a rule-based mock that needs no setup. A setup check (check_ollama.py) recommends a model size that fits the laptop's memory.

Data flows as follows: a batch or PDF enters the UI, the graph passes shared state between agents, the agents call tools, data and memory, escalations pause for a reviewer, and the decision returns to the UI. Decision-making stays rule-based (in tools.decide). The language model only reads and explains: it extracts invoice data, gives the Critic a second opinion, and writes the risk note, the decision reason and the reviewer's case summary (Section 4.9).

### 4.5 Fraud Agent

A three-way match only checks that the documents agree, and some fraud passes that test: a fake vendor that copies a real purchase order, or a genuine vendor's invoice that asks to be paid into a new account. The Fraud Agent runs after matching and adds up risk points. Every point is shown with its reason, so the reviewer sees exactly why an invoice was stopped.

| Signal | How it is detected (risk points) |
|---|---|
| Bank-account change | Pay-to account differs from the account on file in vendor memory (+70). |

| Signal | How it is detected (risk points) |
|---|---|
| Lookalike vendor | Unknown vendor name at least 85% similar to an approved vendor, e.g. 'Acme Suplies' (+60). |
| Amount spike | Total above 3 times the vendor's largest past approved invoice (+40). |
| Unknown vendor / duplicate | Raised by the Matching Agent and counted in the score (+30 each). |
| Round amount | Total is an exact multiple of INR 1,000 (+15). |
| Vendor track record | Share of the vendor's past invoices that were escalated (up to +20). |

A score of 40 or more escalates the invoice. The spike threshold was chosen by testing candidate rules on 200,000 simulated invoices: rules based on the vendor's average misfired on about 2.8% of clean invoices, while 3 times the largest past invoice misfired on about 0.02%.

## 4.6 Human in the loop

Escalated invoices do not simply end with a label. The Human Review node calls LangGraph's interrupt: the graph saves its state to a checkpointer and stops. The invoice appears in the Review queue as a case card. The vendor, amount, risk meter and a short case summary come first: what the problem is, how much money is at stake, the vendor's history, and what to check before deciding. The full findings, invoice text and agent trace sit one click below. The Approve, Hold and Reject buttons carry equal visual weight, so the reviewer is not nudged towards paying a risky invoice. When the reviewer decides, the graph resumes from the checkpoint with that answer, and the decision, reviewer name and note are kept in an audit trail. The same pause and resume also works in the command-line tool (run_cli.py --hitl), which prints the case summary, and in the plain-Python runner used when LangGraph is not installed.

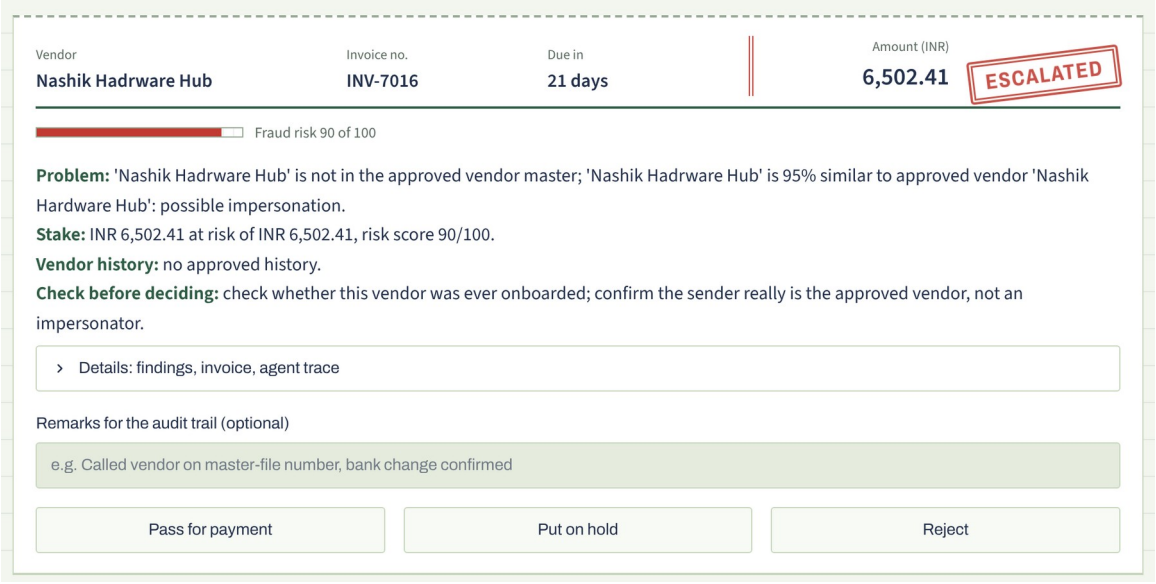



*Figure 1: A case card in the Review queue (real screenshot). A lookalike vendor scored 90/100; the case summary states the problem, the money at stake, the vendor's history and what to check.*

## 4.7 Vendor memory

Each vendor has a long-term profile: its past approved totals, the bank account on file, and how often its invoices were escalated or overridden. The Fraud Agent reads it and the Human Review step writes to it,

so the system learns from people. For example, once a reviewer approves a bank change, the new account becomes the one on file and that vendor's next invoice to it is not flagged. The memory is a JSON file, so it can persist across sessions.

### 4.8 PDF upload and tests

Users can upload text-based PDF invoices, which go through the same agents against the same master data and memory. The app can also export any batch invoice as a sample PDF; uploading one that was already processed is caught as a duplicate. With the mock backend only this demo layout parses; other layouts need the Ollama backend for LLM extraction. A pytest suite of 24 end-to-end tests covers the pipeline, the fraud rules, pause and resume in both runners, memory persistence, PDF handling, the grounding check, the no-new-numbers guards and Ollama model-name matching.

### 4.9 The local language model and its guardrails

A language model is useful for reading messy text and for explaining things in plain words, but it can also invent facts. The design therefore lets the model help in four places and puts a hard check behind each one. The rules decide; the model explains.

| Where the model helps | Guardrail behind it |
|---|---|
| Extraction: reads invoice text into the typed schema. | The Critic checks the arithmetic and that every key field appears in the document. A failure means a retry with the parser. |
| Critic second opinion: flags values in the wrong field, a wrong vendor or bank account, or missing lines. | Advisory only. It can trigger one retry but never escalates an invoice. Malformed replies are ignored. |
| Fraud risk note: explains the risk score in two sentences and says what to verify. | The rules set the score. A note containing any number not in the facts is replaced by the rule-based note. |
| Case summary: problem, stake, vendor history and what to check, for each escalated invoice. | Same no-new-numbers guard. If the model is slow, down or invents a figure, a template summary is used. |

Setup is kept simple for a student laptop. check_ollama.py reports the RAM and installed models and suggests llama3.2, qwen2.5:3b or qwen2.5:1.5b. The app matches a short model name to the installed tag (for example llama3.2 to llama3.2:3b). If Ollama is not running or a model is missing, the app says what to fix and falls back to the rule-based mode, so the demo always runs.

### 4.10 Interface

The interface was redesigned to look like an accounts ledger rather than a generic dashboard: ruled paper, rubber-stamp decisions (Approved, Hold, Escalated, Rejected), a control-totals row for the batch, and a seven-step pipeline strip that ticks off each agent as it runs. Colour is kept for decisions and risk, so the eye goes to what needs attention. Only the presentation changed; the agent logic is the same.

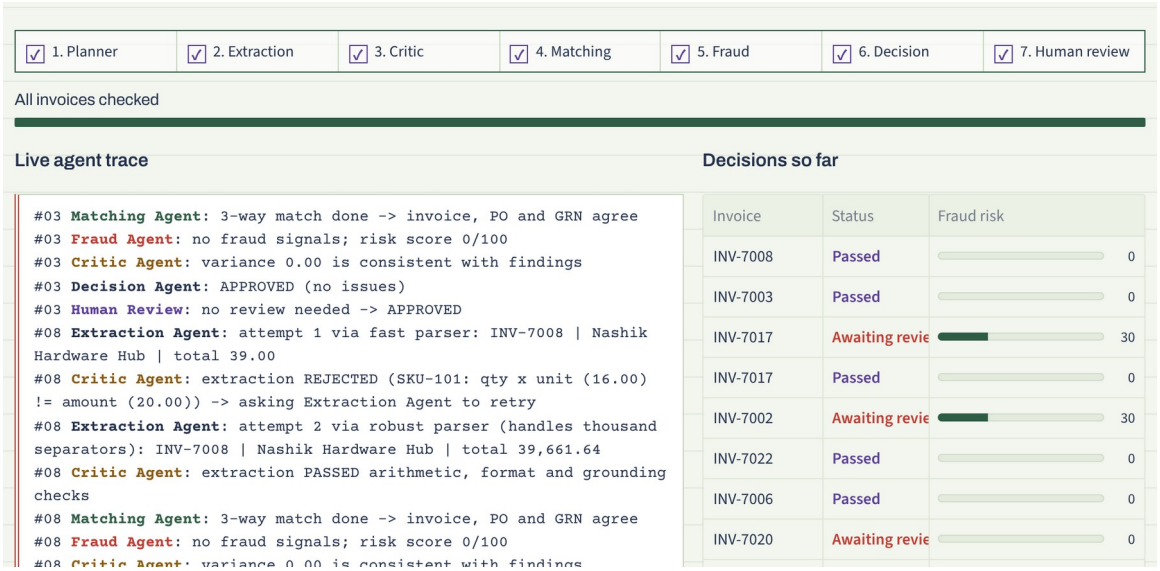



*Figure 2: The Batch tab after a run (real screenshot): the pipeline strip, the live agent trace and each decision with its fraud risk.*

## 5. Demo Walkthrough

1. Generate batch: 24 synthetic invoices, 8 of them with seeded errors (3 of them fraud).
2. Run agents: watch the pipeline strip and the live agent trace.
3. Review queue: read each case summary, then approve, hold or reject the invoices paused for a human.
4. Upload PDF: download a sample invoice as a PDF, upload it, and watch the agents catch it as a duplicate.

Live trace for one invoice (real output):

```
Extraction Agent: attempt 1 via fast parser: INV-7009 | total 1.00
Critic Agent: extraction REJECTED (qty x unit != amount) -> retry
Extraction Agent: attempt 2 via robust parser: INV-7009 | total 1,734,135.13
Fraud Agent: AMOUNT_SPIKE (8.1x vendor's largest invoice); risk score 40/100
Decision Agent: ESCALATED (fraud risk score 40 >= 40) -> Review queue
```

The trace shows the Critic rejecting a misread total (1.00), the retry recovering the correct total, and the Fraud Agent noticing that this vendor has never billed anything close to that amount before.

## 6. Results

Measured on 24 synthetic invoices: 8 with seeded errors (3 of them fraud) and 16 clean.

| Result | Score | Meaning |
|---|---|---|
| Seeded errors caught | 8 / 8 | Every seeded error was detected. |
| Issue types identified | 8 / 8 | Each error was classified with the correct issue type. |
| Fraud cases escalated | 3 / 3 | Lookalike vendor, bank change and amount spike were all stopped for a human. |
| Clean invoices wrongly flagged | 0 / 16 | No false positives on clean invoices. |

| Result | Score | Meaning |
|---|---|---|
| Amount at risk flagged | INR 0.99 lakh | Equal to the seeded total. |
| Large-scale check | 99.98% | 37,200 invoices over 300 seeds and three batch sizes; 8 misses, all rare amount-spike edge cases. |
| Automated tests | 24 / 24 pass | pytest: pipeline, fraud rules, pause and resume, memory, PDF upload, grounding check, LLM guards. |

The Critic loop caught misread amounts, and the retry fixed them before matching. These high scores reflect rule-based matching on known data, not real-world accuracy: the answer key is known because the data is synthetic and the matching rules are deterministic. These figures are for the default rule-based mode. With Ollama the decision rules are the same, but a small model can misread fields, and the Critic loop is what catches those errors.

## 7. Limitations and Future Scope

| Limitations | Future scope |
|---|---|
| Synthetic data and one fixed invoice layout | OCR for scanned PDFs and photos |
| Rule-based matching scores optimistically | ERP connectors such as SAP and Tally |
| No OCR for scanned PDFs, no ERP link or login | Benchmark LLM extraction against the parser |
| Time savings are estimates, not measurements | Learn fraud weights from reviewer decisions |
| Ollama runs are slower; small models can misread fields | Role-based approvals and MCP tools |

The biggest next step is moving to real invoices, using OCR together with an LLM extractor.

## 8. Conclusion

A small team of single-purpose agents, coordinated by a state graph and checked by a Critic, can run an invoice three-way match end to end, retry its own extraction errors, spot fraud that matching alone misses, and explain every decision. Risky invoices pause for a person with a clear case summary, and the system learns from what that person decides. A local language model makes the output easier to act on, while hard checks keep it from changing a decision or inventing a figure. The prototype performs almost perfectly on its synthetic test set, and the honest next step is to test it on messy real-world invoices.

## 9. Project Links

**GitHub repository:** https://github.com/Saafalya/invoice_agents

**Clone URL:** `https://github.com/Saafalya/invoice_agents.git`